The Even-Index Encoding for Rectilinear Polygons

Simplicity, run time, and memory versus the full-vertex list

Wai-Shing Luk

physdes

October 26, 2026

Section 1

Outline

Outline

1. Outline
2. Introduction
3. Representation
4. Comparison
5. Monotone Hulls
6. Recursive Decomposition
7. Verification
8. Conclusion

Section 2

Introduction

Rectilinear Geometry in Physical Design

- VLSI layouts are overwhelmingly **Manhattan**: every edge is horizontal or vertical.
- Rectilinear polygons model **obstacles**, **regions**, and **layout fracturing** patterns [1], [2].
- Coordinates are **integers**; rounding is not merely inaccurate but illegal.
- The polygons of interest are small: $n \leq 100$ vertices.
- At these sizes a quadratic algorithm is not merely acceptable: with small constants it can beat a sub-quadratic method. Clarity and exactness dominate.

The Representation Question

Input. A simple rectilinear polygon P.

- Full boundary: n vertices, $2n$ coordinates.
- Even-index: $m = n/2$ stored points; the omitted vertex is (x_{i+1}, y_i).
- The encoding is reversible, but what does the halving actually buy?

Contributions.

1. A compact encoding, its canonical form, and the reconstruction rule.
2. An exact cost model for storage, passes, and expansion.
3. A C++20 benchmark and a full-vertex convex baseline.
4. A three-step rectangle decomposition in $O(m^2)$.

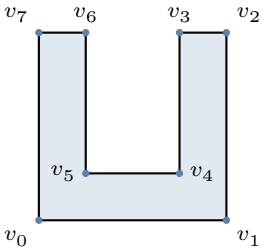
Headline Results

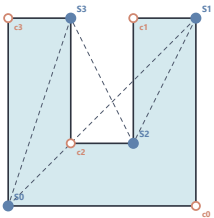
- **Memory:** 2.00× fewer bytes at the largest size; 1.91× at $n = 64$.
- **Bounding box:** 1.93–2.00× faster.
- **Area / canonicality:** only about 1.3×, not 2×; per-iteration work does not halve.
- **Expansion:** 7–13× the full area pass at $m = 8192$.
- **Full-vertex convex cut:** 2–12× slower; 12× at $n = 260$.
- **Source size:** 268 versus 220 NBNC for canonical output; no advantage.

Section 3

Representation

Full Boundary and Even-Index Encoding

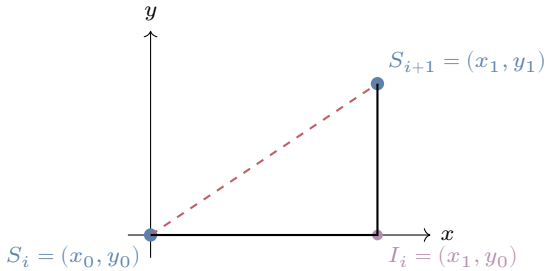




Definition. For a full boundary $V = (v_0, \dots, v_{n-1})$ with n even, the even-index encoding is $S = (v_0, v_2, \dots, v_{n-2})$.

- The odd-index vertices are implicit; only $m = n/2$ points are stored.
- The represented polygon is recovered by the x-first rule below.

x-First Expansion



Definition (x-first expansion). For consecutive stored points $S_i = (x_0, y_0)$ and $S_{i+1} = (x_1, y_1)$, insert

$$I_i = (x_{i+1}, y_i)$$

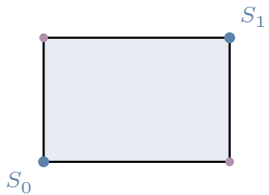
before S_{i+1}, including the closing pair $S_{m-1} \to S_0$.

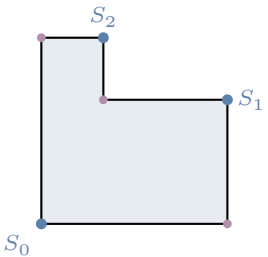
- The diagonal chord $S_i S_{i+1}$ is **not** a boundary edge.
- The y-first form (x_0, y_1) reconstructs a different polygon.

Reconstruction

- The full boundary is recovered by inserting $I_i = (x_{i+1}, y_i)$ between consecutive stored points.
- The first edge is horizontal, so the x-first rule is exact.
- The y-first form (x_0, y_1) reconstructs a different polygon.
- Reducedness is essential: a 180° vertex breaks the rule.
- Simplicity is a global condition on the boundary.

Minimal Encodings





- **2 stored points** represent a rectangle: $E(S)$ has four corners.
- **3 stored points** represent a monotone / L-shaped piece: six corners.
- These are not degenerate cases; they are the base cases of the decomposition.

Canonical form. For a simple input polygon, every cyclic consecutive pair of stored points differs in both coordinates; every simple-polygon encoding is canonical. Canonicality is local and does not imply simplicity.

Canonical Form

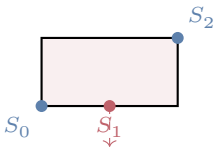
Definition. A stored sequence $S = (s_0, \dots, s_{m-1})$ is **canonical** if

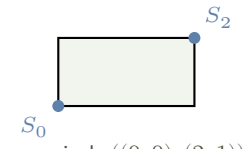
1. $m \geq 2$;
2. no cyclic consecutive points coincide;
3. if $m = 2$, the two points are distinct (a shared axis is allowed);
4. if $m \geq 3$, no point lies strictly between its neighbours on a shared axis.

```
if |S| < 2: return false
if any cyclic consecutive duplicate: return false
if |S| == 2: return true
for each i (cyclic):
    p, q = prev(i), next(i)
    if p.y == s_i.y and x_i strictly between x_p, x_q: return false
    if s_i.x == q.x and y_i strictly between y_p, y_q: return false
return true
```

- Consecutive duplicates are rejected first.
- Non-adjacent duplicates are allowed: canonical, but not simple.

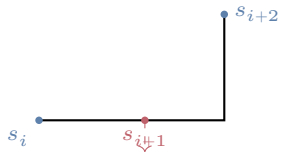
Betweenness Is Redundant



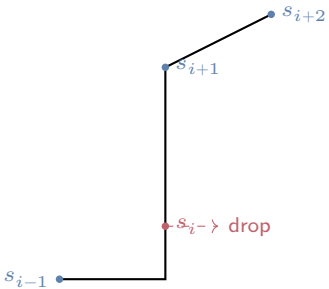


- $S = ((0,0),(1,0),(2,1))$ has S_1 strictly between S_0 and S_2 on the bottom edge.
- Deleting S_1 preserves the boundary, perimeter, and area: the represented polygon is unchanged.
- The two lists have the **same area and perimeter**, so area-only (or perimeter-only) tests cannot detect the redundancy; canonicalization removes it.
- A *spike* is the complementary case: a shared-axis point that is not between its neighbours. Deleting it would shorten the boundary, so it is not redundant.

Canonicalization



same y: drop the second



same x: drop the first

- Remove cyclic consecutive duplicates first.
- A same-y pair drops the second point when it lies strictly between its neighbours; a same-x pair drops the first analogously.
- A work queue revisits only the neighbours of a removed point: $O(m)$.
- The final `is_canonical` check is a safety net.

Perimeter. Over stored pairs,

$$P(S) = \sum_{i=0}^{m-1} (|x_{i+1} - x_i| + |y_{i+1} - y_i|);$$

the two terms are exactly the two boundary legs between S_i and S_{i+1}.

Signed area in the implementation's form:

$$A(S) = v_1^x v_1^y + \sum_{i=2}^{m-1} v_i^x (v_i^y - v_{i-1}^y), \qquad v_i = s_i - s_0.$$

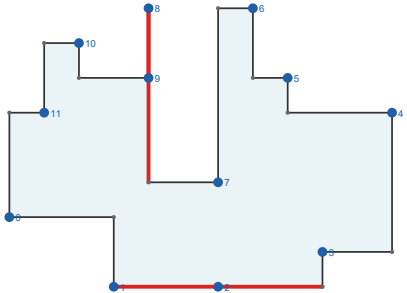
- **Lemma.** This equals the signed area of the reconstructed boundary.
- $A > 0$: anticlockwise; $A < 0$: clockwise.
- Canonicalization preserves both P and A.
- The benchmark uses this formula on both representations, not a different one.

Creation Outputs Are Not Canonical

rpolygon_xmono: hand-written 12-point fixture

is_canonical = False | red = consecutive same-orientation edges

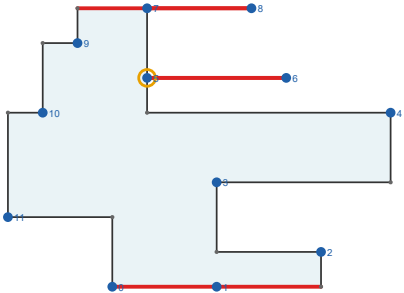
violation 1: (-3, -4) -> (0, -4) -> (3, -4)
violation 2: (-2, -1) -> (-2, 4) -> (-2, 2)



rpolygon_ymono: hand-written 12-point fixture

is_canonical = False | red = consecutive same-orientation edges

violation 1: (-3, -4) -> (0, -4) -> (3, -4)
violation 2: (-2, 2) -> (2, 2) -> (-2, 2)
violation 3: (-2, 4) -> (1, 4) -> (-4, 4)



- Raw creation orderings contain consecutive collinear triples.
- Canonicalization removes the redundant points.

Section 4

Comparison

Simplicity

Aspect	Full vertex list	Even-index encoding
Stored records	n points	$m = n/2$ points
Omitted vertices	none	m, recovered by one rule
Canonicality check	duplicate and collinear scan over n	local betweenness scan over m
Rectangle base case	4 vertices	2 points
L-shape base case	6 vertices	3 points
Axis convention	none	fixed x-first
GDSII interchange	direct	expand first

- The base-case saving is real.
- The axis convention is a price, not a code-size advantage.

Run Time: Counting versus Timing

Pass	Full list	Even-index	Trip ratio	Measured
Storage records	n	m	2	$2.00\times$ bytes
Bounding box	n	m	2	1.93–$2.00\times$
Area	$n-1$ terms	$m-1$ terms	≈ 2	$\approx 1.3\times$
Canonicality	n tests	m tests	2	$\approx 1.3\times$
Perimeter	n edges	m pairs	1 arithmetic	not measured
Expansion	—	$O(m)$	—	7–$13\times$
Cut	$O(n^2)$	$O(m^2)$	4 pairs	not measured

- Trip count halves; timing follows only where per-iteration work is unchanged.
- Both sides use the same telescoping area formula.
- The asymptotic order is unchanged.

Measured Throughput

m	n	area ratio	bbox ratio	canon ratio
128	256	1.41	1.93	1.30
512	1024	1.72	1.95	1.32
2048	4096	1.34	1.98	1.35
8192	16384	1.33	2.00	1.34

- Median of 7 trials, MSVC release, Intel i9-13900H; ratio = full/even.
- Area is about $1.3\times$ at the largest size, not $2\times$.
- Expansion at $m = 8192$: 138–183 μs versus 14–19 μs for the full area pass (7–$13\times$).
- Every row uses the same polygon for both representations.

Memory

- 8 bytes per point (two 32-bit integers); 32-byte header.

$$B_{\text{even}}(n) = 32 + \left(\frac{n}{2} - 1\right) 8, \qquad B_{\text{full}}(n) = 32 + (n-1)8.$$

n	even bytes	full bytes	ratio
64	280	536	1.91
256	1048	2072	1.98
1024	4120	8216	1.99
4096	16408	32792	2.00
16384	65560	131096	2.00

- The ratio tends to two; measured allocations agree.
- GDSII stores 8 bytes per vertex, so the payload is halved.
- Expansion adds n points temporarily: the saving is in steady state.

Source Size

Non-blank, non-comment lines (NBNC), C++20:

Scope	Even-index	Full-vertex
Decomposition logic	185	187
Canonicalization	83	33
Total, canonical output	268	220

- The decomposition logic is almost identical.
- The even-index canonicalizer is more elaborate.
- Source size does not favour the encoding; its advantage is memory and time.

Full-Vertex Convex Baseline

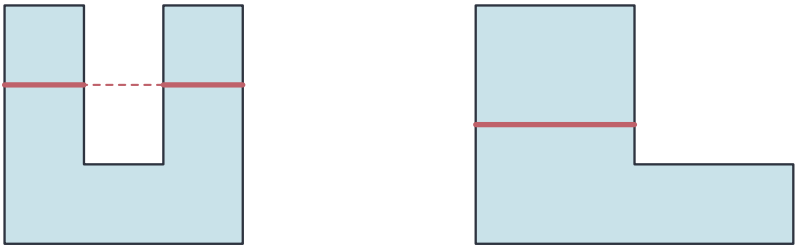
- Stores all n vertices; same linked list and $O(1)$ split as the even-index version.
- Tests orthogonal convexity globally (x and y monotonicity); an L-shape has a reflex vertex but is convex, so a local reflex test is insufficient.
- Cuts at the first reflex vertex along an interior axis.

Fixture	n	pieces (even/full)	full cut (μs)	ratio
Twelve-point	24	3 / 5	10.8	2.5×
Comb $k = 2$	12	3 / 3	5.1	2.0×
Comb $k = 8$	36	9 / 9	25.3	3.1×
Comb $k = 32$	132	33 / 33	213.6	7.2×
Comb $k = 64$	260	65 / 65	788.3	11.9×

- Same polygon per row; the gap grows with size.
- Remaining cost: global convexity test plus $2\times$ point count.

Section 5

Monotone Hulls



Definition. A rectilinear polygon is x-monotone if every vertical line meets it in a single interval; y-monotone is defined symmetrically.

Lemma. A rectilinear polygon is orthogonally convex if and only if it is both x-monotone and y-monotone.

Monotone Hull Algorithm

- Let p, c, n be the previous, current, and next stored vertices.
- A vertex is removed when monotonicity is violated and the turn has the wrong sign:

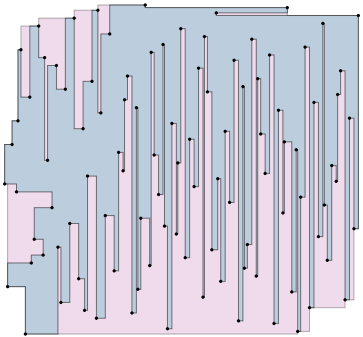
$$\Delta = (y_c - y_p)(x_n - x_c).$$

- A circular doubly-linked list makes removal $O(1)$.
- Two passes: minimum to maximum, then maximum to minimum.

```
for c in chain:
    if dir(c) > dir(n) or dir(p) > dir(c):
        if wrong_sign(Delta):
            remove(c); c = prev(c)
        else:
            c = next(c)
    else:
        c = next(c)
```

Complexity. Each sweep is $O(m)$; the hull's internal restart scan is $O(m^2)$ worst case.

Hull Composition



- x-hull then y-hull yields the rectilinear convex hull.
- Hulls also give monotone enclosures and preprocessing for decomposition.
- The output is checked for canonicality and monotonicity.

Section 6

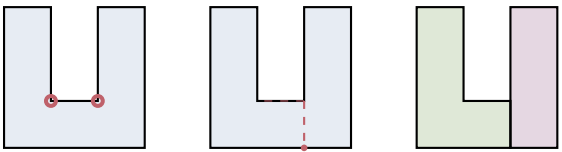
Recursive Decomposition

Three-Step Decomposition



- **Step 1: convex cut.** Remove concavities; simplify the remaining geometry.
- **Step 2: explicit 270°.** Resolve reflex corners that are stored vertices.
- **Step 3: implicit 270°.** Resolve reflex corners introduced by expansion.
- Steps 2 and 3 repeat until neither type remains; the final pieces are rectangles, suitable for layout fracturing and DRC [3], [4].

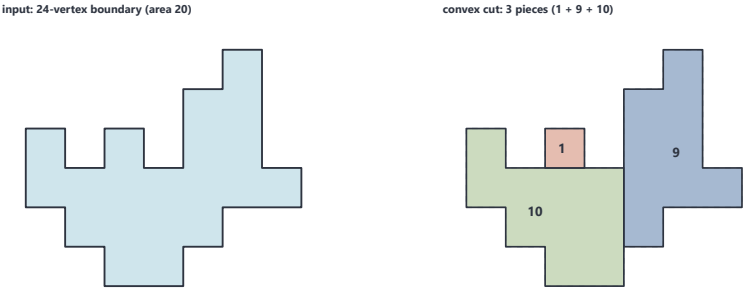
Cut Primitive



(a) concave vertices (b) minimum cut (c) split and recurse

- Find a concave vertex: a direction reversal with the correct area sign.
- Project v onto each edge; choose the nearest visible edge and insert the intersection point.
- Splice the new node, split the circular list, and recurse on both pieces.

Convex Decomposition Example



- Three orthogonally convex pieces.
- Areas $1, 9, 10$; total area 20.

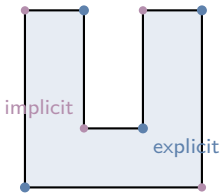
Base Cases and Recursion

Base cases.

- 2 stored points: a rectangle; stop.
- 3 stored points: a monotone / L-shaped piece; stop in the convex variant.
- The explicit variant has no 3-point base case; it continues until no concave vertex remains.

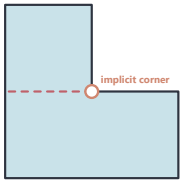
```
cut(v1):
    v2, v3 = v1.next, v1.next.next
    if v3 == v1:
        return [v1, v2]
    if triangle_base and v3.next == v1:
        return [v1, v2, v3]
    c = find_concave(v1)
    if c is None:
        return all_vertices(v1)
    vmin, vertical = find_min_dist(c)
    new = insert_cut(c, vmin, vertical)
```

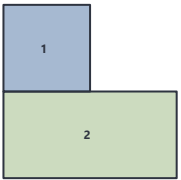
Explicit and Implicit 270° Vertices



- **Explicit $270°$:** the reflex corner is a stored even-index vertex.
- **Implicit $270°$:** the reflex corner is created by the x-first expansion.
- The same compressed U-shape contains both kinds.
- The convex cut handles the shape; the explicit and implicit cuts handle the corner types.

Implicit Cut of an L-Shape



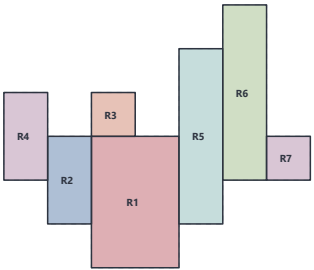


- The implicit corner $(1, 1)$ is reflex.
- A horizontal cut yields rectangles of areas 2 and 1.

Rectangle Output and Canonicality

- A rectangle piece has **2 stored points**: two opposite corners.
- A canonical 2-point list only needs distinct points; a non-degenerate rectangle also needs distinct x and y.
- The three-step algorithm outputs rectangles; the tests assert the two-point form, distinct coordinates, canonicality, and the area.
- For larger intermediate pieces, canonicality and orthogonal convexity are checked on the expanded boundary.

Rectangle Decomposition



- Seven rectangles, areas $1, 1, 2, 2, 4, 4, 6$.
- Total area 20.

Correctness Invariants and Complexity

Invariants.

1. **Area:** the sum of signed piece areas equals the signed area of the input.
2. **Rectangle output:** the three-step algorithm returns two-point rectangles.
3. **Canonicality:** local. Orthogonal convexity is checked on the expanded boundary.

Complexity.

operation	cost
canonicality check	$O(m)$
canonicalization	$O(m)$
concave-vertex scan	$O(m)$
nearest-edge scan	$O(m)$
insert / split	$O(1)$
total decomposition	$O(m^2)$

For $n \leq 100$, quadratic time is preferable to a complicated sub-quadratic

Section 7

Verification

C++20 (xmake)

- `xmake build test_recti`
- `xmake run test_recti`
- **147 test cases**
- **1431 assertions**
- **0 failures**

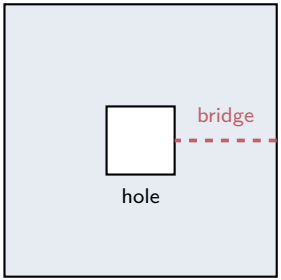
Benchmarks

- `xmake build bench_repr` / `xmake run bench_repr`
- `xmake build bench_cut` / `xmake run bench_cut`
- MSVC release, Intel i9-13900H
- Canonicalization processes the creation ordering before the canonicality check; raw creation is only an ordering, not a polygon.
- The tests assert the output class: canonicality, orthogonal convexity, and rectangle

Limitations and Threats to Validity

1. **Canonicality.** It is a local property of the encoding; simplicity is an assumption on the input.
2. **Synthetic inputs.** The evaluation uses generated point sets and hand-built fixtures, not foundry data; the claims are algorithmic, not signoff.
3. **Quadratic scaling.** Decomposition is $O(m^2)$; canonicalization is $O(m)$. For $n \leq 100$ the quadratic algorithm is often faster, and sub-quadratic methods are not addressed here.
4. **No optimality claim.** The three-step algorithm returns a rectangle partition, not a minimum-cardinality one.
5. **Robustness.** Integer arithmetic is exact; degenerate input handling beyond the stated invariants is outside the present scope [5].
6. **Benchmark scope.** One machine, one compiler, and one fixture family; the area ratio is indicative, while the byte and bounding-box ratios are stable.

Polygons with Holes



- A bridge cut from the outer boundary to a hole merges the boundaries into a single cycle; a zero-width bridge gives a weakly simple boundary, a narrow bridge gives a simple one.
- The bridge is traversed in both directions, so the area is unchanged.
- The even-index encoding and the cut algorithms then apply; repeated bridge vertices are handled by the degenerate split.
- This is standard boundary bookkeeping, not specific to the even-index

Section 8

Conclusion

Contributions and Open Problems

Contributions.

- A canonical form with a redundant-vertex criterion.
- An $O(m)$ canonicalization that preserves perimeter and area.
- An exact cost model and C++20 measurements: 2.00× bytes and bounding box, about 1.3× area, 7–13× expansion.
- A full-vertex convex baseline: valid pieces, 2–12× slower (12× at $n = 260$).
- A three-step rectangle decomposition: convex decomposition, explicit 270° handling, and implicit 270° handling.
- Output-class tests and C++20 verification (147 cases, 1431 assertions).

Open problems.

- A formal proof that the compressed predicate characterizes orthogonal convexity.
- Minimum-cardinality rectangle partitioning.
- Cross-language differential testing on random polygons.

Q&A

- Code: `physdes-cpp` under the `luk036` organization.
- Thank you.

References I

[1] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA: Kluwer Academic Publishers, 1999.

[2] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht: Springer, 2011.

[3] A. B. Kahng, C.-H. Park, X. Xu, and H. Yao, "Layout decomposition for double patterning lithography," in *Proc. IEEE/ACM int. Conf. Computer-aided design (ICCAD)*, 2008, pp. 465–472. doi: 10.1109/ICCAD.2008.4681616.

[4] L. W. Liebmann, "Layout impact of resolution enhancement techniques: Impediment or opportunity?" in *Proc. Int. Symp. Physical design (ISPD)*, 2003, pp. 110–117. doi: 10.1145/640000.640026.

[5] H. Edelsbrunner and E. P. Mücke, "Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms," *ACM Trans. Graph.*, vol. 9, no. 1, pp. 66–104, 1990.

References II